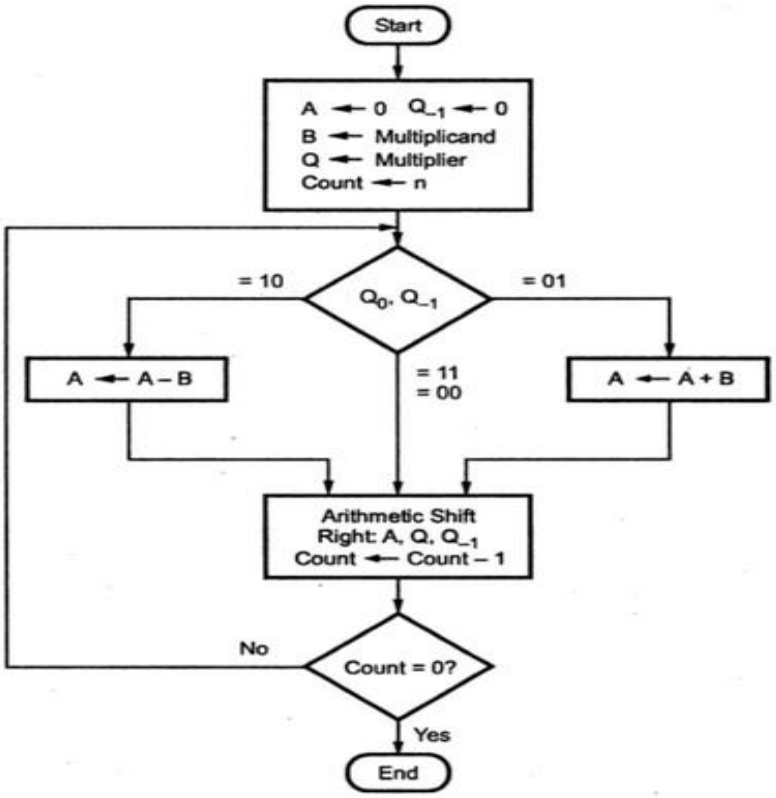
Multiplication

Booth Algorithm:

1. Set the Multiplicand and Multiplier binary bits as M and Q, respectively.
2. Initially, we set the Aand Q_{-1}registers value to 0.
3. Count represents the number of Multiplier bits (Q), and it is a sequence counter that is continuously decremented till equal to the number of bits (n) or reached to 0.
4. On each cycle of the booth algorithm, Q_0 and Q_{-1} bits will be checked on the following parameters as follows:
    i. When two bits Q_0 and Q_{-1} are 00 or 11, we simply perform the arithmetic shift right operation (ashr) to the partial product A. And the bits of Qn and Q_{n+1} is incremented by 1 bit.
    ii. If the bits of Q_0 and Q_{-1} is shows to 01, the multiplicand bits (M) will be added to the A (Accumulator register). After that, we perform the right shift operation to the AC and Q_0 bits by 1.
    iii. If the bits of Q_0 and Q_{-1} is shows to 10, the multiplicand bits (M) will be subtracted from the A (Accumulator register). After that, we perform the right shift operation to the AC and Q_0 bits by 1.
5. The operation continuously works till we reached Count bit<1 in the booth algorithm.
6. Results of the Multiplication binary bits will be stored in the A and Q_0 registers.



Example :

Both Positive (5×4)

Multiplicand (B) ← 0 1 0 1 (5)			Multiplier (Q) ← 0 1 0 0 (4)	
Steps	**A**	**Q**	Q_{-1}	**Operation**
	0 0 0 0	0 1 0 **0**	**0**	Initial
Step 1 :	0 0 0 0	0 0 1 **0**	**0**	Shift right
Step 2 :	0 0 0 0	0 0 0 **1**	**0**	Shift right
Step 3 :	1 0 1 1	0 0 0 **1**	**0**	A ← A – B
	1 1 0 1	1 0 0 **0**	**1**	Shift right
Step 4 :	0 0 1 0	1 0 0 **0**	**1**	A ← A + B
	0 0 0 1	0 1 0 **0**	**0**	Shift right
Result :	0 0 0 1	0 1 0 0	=	+ 20

CASE 2 : Negative Multiplier (5 × – 4)

Multiplicand(B) ← 0 1 0 1 (5)		Multiplier (Q) ← 1 1 0 0 (– 4)		
Steps	**A**	**Q**	Q_{-1}	**Operation**
	0 0 0 0	1 1 0 **0**	**0**	Initial
Step 1 :	0 0 0 0	0 1 1 **0**	**0**	Shift right
Step 2 :	0 0 0 0	0 0 1 **1**	**0**	Shift right
Step 3 :	1 0 1 1	0 0 1 **1**	**0**	A ← A – B
	1 1 0 1	1 0 0 **1**	**1**	Shift right
Step 4 :	1 1 1 0	1 1 0 **0**	**1**	Shift right
Result :	1 1 1 0	1 1 0 0	= – 20 (2's complement of 20)	

CASE 3 : Negative Multiplicand (– 5 × 4)

Multiplicand(B) ← 1 0 1 1 (– 5)		Multiplier(Q) ← 0 1 0 0 (4)		
Steps	**A**	**Q**	Q_{-1}	**Operation**
	0 0 0 0	0 1 0 **0**	**0**	Initial
Step 1 :	0 0 0 0	0 0 1 **0**	**0**	Shift right
Step 2 :	0 0 0 0	0 0 0 **1**	**0**	Shift right
Step 3 :	0 1 0 1	0 0 0 **1**	**0**	A ← A – B
	0 0 1 0	1 0 0 **0**	**1**	Shift right
Step 4 :	1 1 0 1	1 0 0 **0**	**1**	A ← A + B
	1 1 1 0	1 1 0 **0**	**0**	Shift right
Result :	1 1 1 0	1 1 0 0	= – 20 (2's complement of 20)	

CASE 4 : Both Negative (– 5 × – 4)

Multiplicand(B) ← 1 0 1 1 (–5) Multiplier(Q) ← 1 1 0 0 (– 4)				
Steps	**A**	**Q**	Q_{-1}	**Operation**
	0 0 0 0	1 1 0 0	0	Initial
Step 1 :	0 0 0 0	0 1 1 0	0	Shift right
Step 2 :	0 0 0 0	0 0 1 1	0	Shift right
Step 3 :	0 1 0 1	0 0 1 1	0	A ← A – B
	0 0 1 0	1 0 0 1	1	Shift right
Step 4 :	0 0 0 1	0 1 0 0	1	Shift right
Result :	0 0 0 1	0 1 0 0 = + 20		

Multiplication using Recoded Multiplier method:

Example 2.5 : *Recode the multiplier 1 0 1 1 0 0 for Booth's multiplication.*

Solution :

Implied zero

1	0	1	1	0	0	0	Multiplier
– 1	+ 1	0	– 1	0	0		Recoded multiplier

Example 2.6 : *Recode the multiplier 0 1 1 0 0 1 for Booth's multiplication.*

Solution :

Implied zero

0	1	1	0	0	1	0	Multiplier
+ 1	0	– 1	0	+ 1	– 1		Recoded multiplier

The Fig. 2.22 shows the Booth's multiplication. As shown in the Fig. 2.22,

Example :

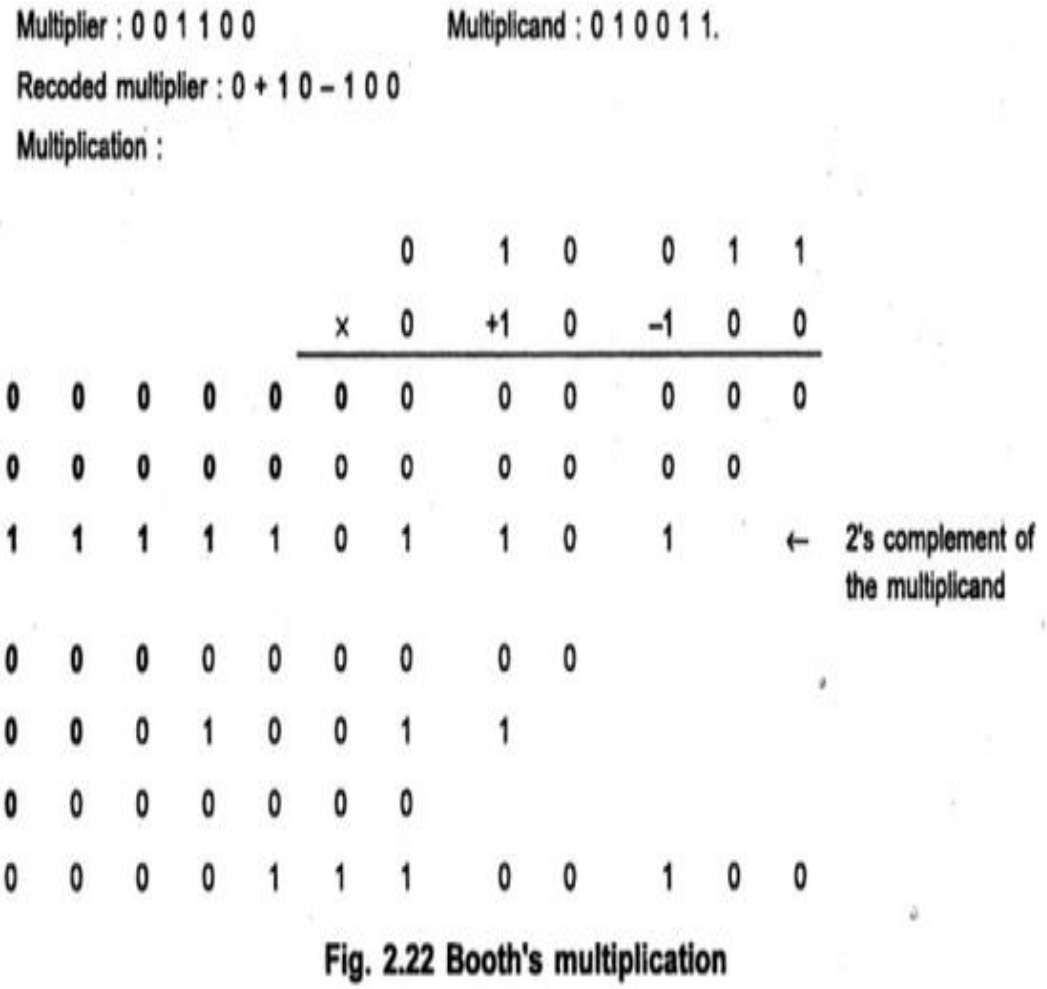


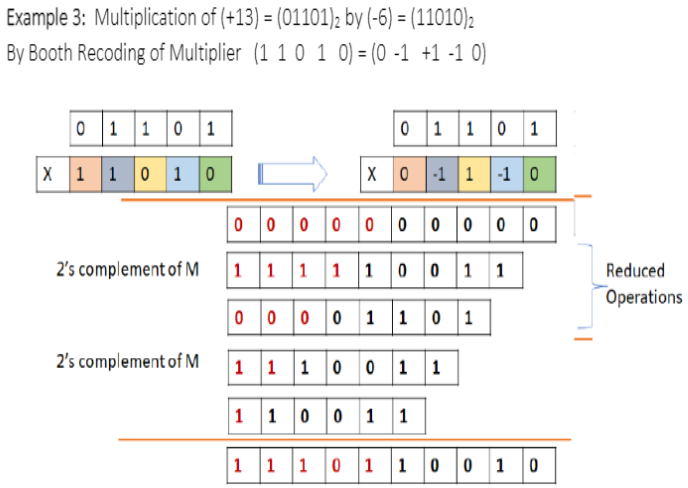
Fig. 2.22 Booth's multiplication

Fast Multiplication (or) bit pair recoding (or) Modified booth algorithm

- To speed up the multiplication process in the Booth's algorithm a technique called Bit-pair recording is used.
- It is also called as modified Booth's algorithm.
- It halves the maximum number of summands.
- i.e., the booth's algorithm may need the summation of each steps and number of steps required in Booth's algorithm are equal to length of multiplier in bits.
- So the bit pair recording halves the maximum number of summations. Hence it achieves faster multiplication.

Multiplier bit pair		Bit pair recoded **multiplier** at position i
i	i+1	
0	-1	-1
0	+1	+1
-1	0	-2
+1	0	+2
-1	+1	-1
+1	-1	+1

Example: Multiply 1101 * 11010 using bit pair recoding algorithm



By Bit Pair Booth Recoding of Multiplier

(0 -1 +1 -1 0) → By bit pairing 0 [-1 +1][-1 0] → 0 $[-2^1 + 2^0]$ $[-2^1 + 0]$ → 0 [-1] [-2]

Now the Multiplicand is multiplied by Bit pair recoded multiplier (0, -1, -2)

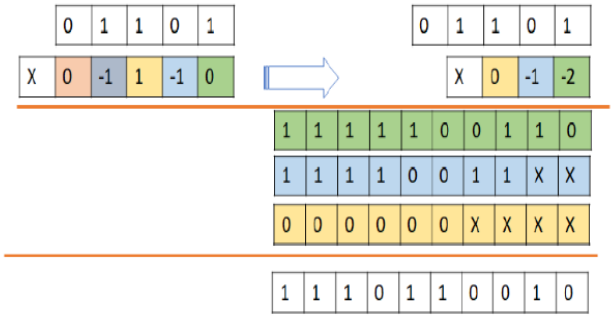


Figure 3.24: Example for Multiplication using Bit Pair recoded of Multiplier

Division

Restoring Algorithm:

Register Q is used to contain the quotient, and register A is used to contain the remainder. Here, the divisor will be loaded into the register M, and n-bit divided will be loaded into the register Q. 0 is the starting value of a register. The values of these types of registers are restored at the time of iteration. That's why it is known as restoring.

Step 1: In this step, the corresponding value will be initialized to the registers, i.e., register A will contain value 0, register M will contain Divisor, register Q will contain Dividend, and N is used to specify the number of bits in dividend.

Step 2: In this step, register A and register Q will be treated as a single unit, and the value of both the registers will be shifted left.

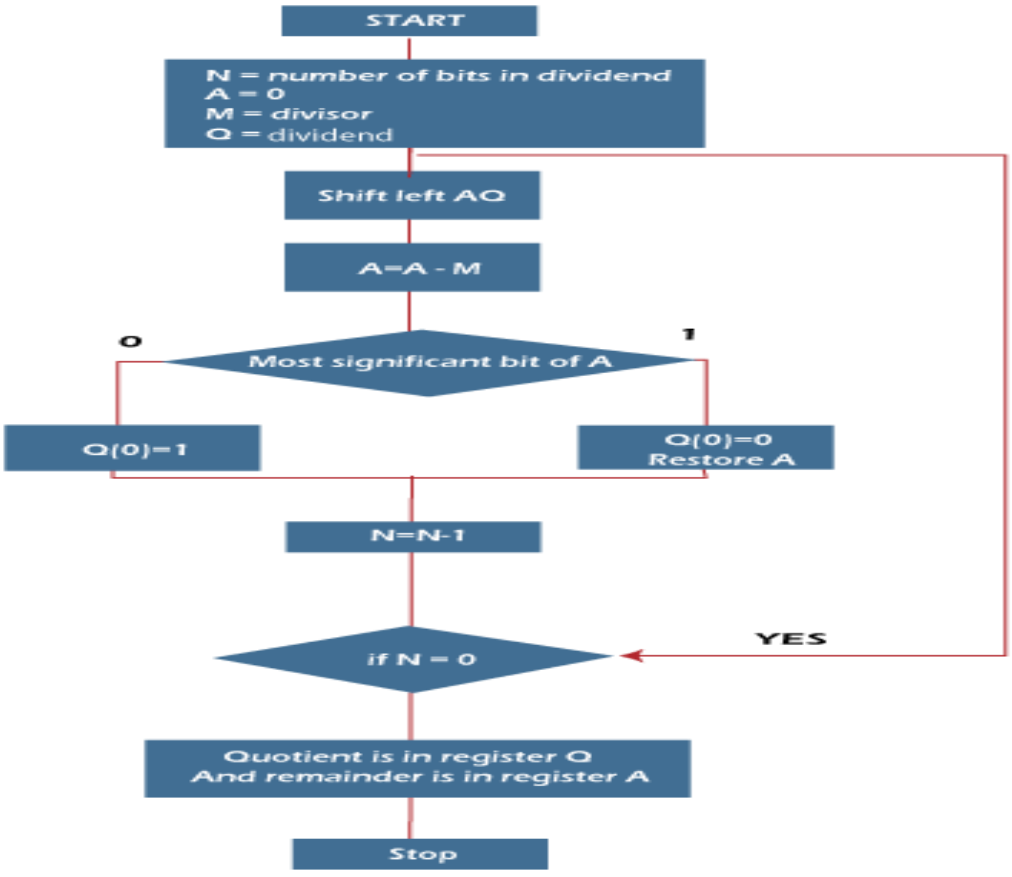
Step 3: After that, the value of register M will be subtracted from register A. The result of subtraction will be stored in register A.

Step 4: Now, check the most significant bit of register A. If this bit of register A is 0, then the least significant bit of register Q will be set with a value 1. If the most significant bit of A is 1, then the least significant bit of register Q will be set to with value 0, and restore the value of A that means it will restore the value of register A before subtraction with M.

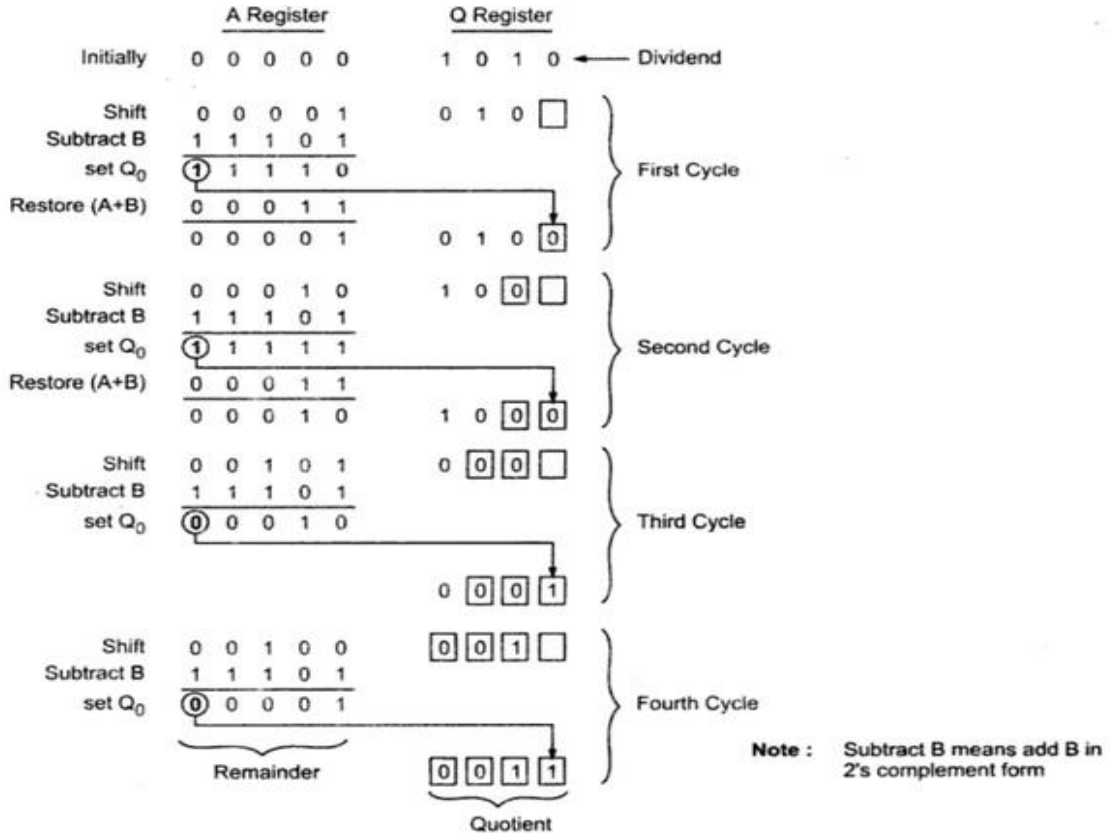
Step 5: After that, the value of N will be decremented. Here n is used as a counter.

Step 6: Now, if the value of N is 0, we will break the loop. Otherwise, we have to again go to step 2.

Step 7: This is the last step. In this step, the quotient is contained in the register Q, and the remainder is contained in register A.



Example : 10 divided by 3



Non-Restoring Algorithm

Corresponding value will be initialized to the registers, i.e., register A will contain value 0, register M will contain Divisor, register Q will contain Dividend, and N is used to specify the number of bits in dividend.

Looking at the above operations we can write following steps for non-restoring algorithm.

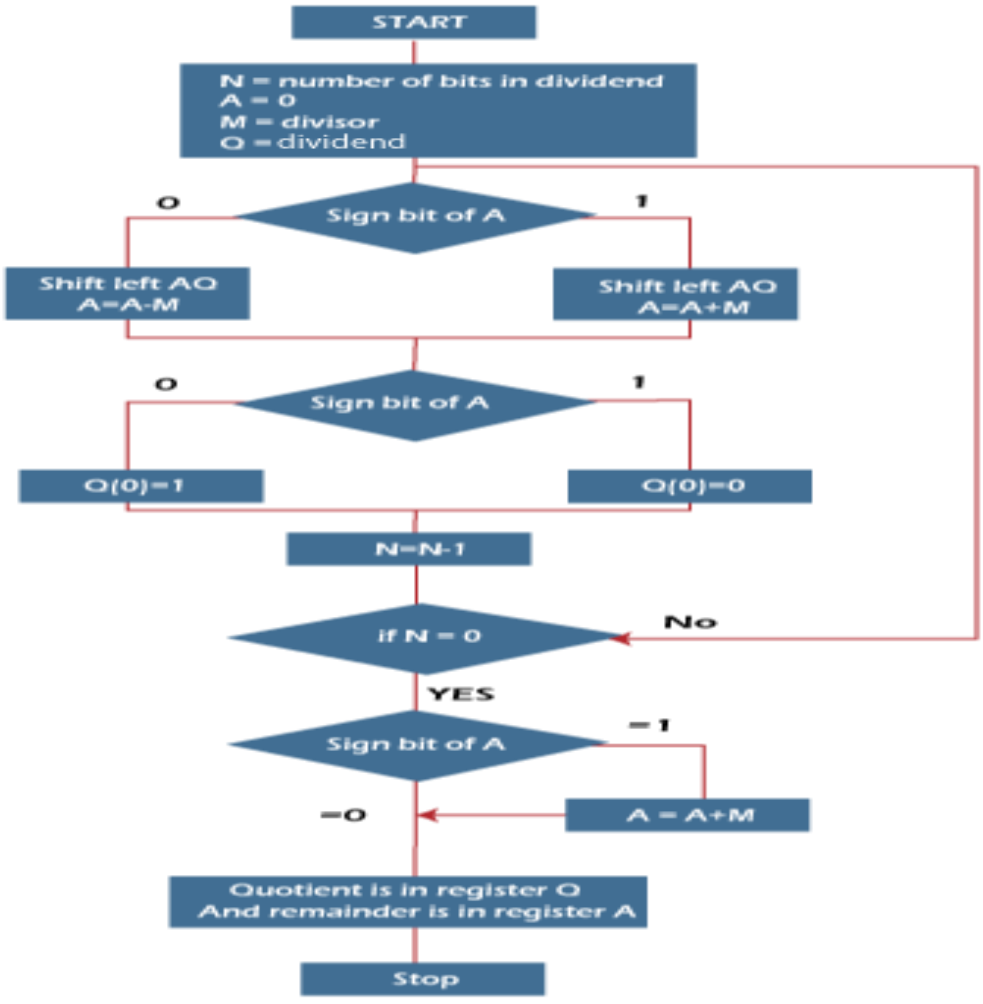
Step 1 : If the sign of A is 0, shift A and Q left one bit position and subtract divisor from A; otherwise, shift A and Q left and add divisor to A.

If the sign of A is 0, set Q_0 to 1; otherwise, set Q_0 to 0.

Step 2 : Repeat steps 1 and 2 for n times.

Step 3 : If the sign of A is 1, add divisor to A.

Note : Step 3 is required to leave the proper positive remainder in A at the end of n cycles.



Example : 10 Divided by 3

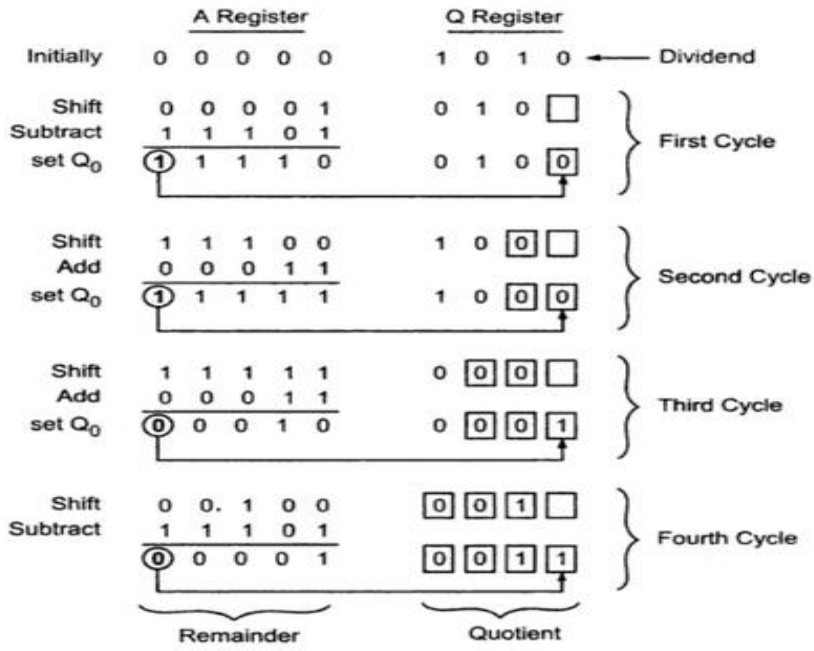


Fig. 2.25 A non-restoring division example

Example 2 : 11 Divided by 5